WAKARA · DOSSIER OFFICIEL DE LIVRAISON

Guide d'installation et d'exploitation

Installer, configurer, déployer, sauvegarder, restaurer et maintenir Wakara

Client	Mairie de Ratoma — Centre d'état civil et de l'identification (CECI Ratoma)
Prestataire	Elhadj Alhassana Camara
Document	04 / 07 — Administrateurs techniques
Version	1.0 — 9 novembre 2026
Application	https://wakara-rtm.vercel.app

Diffusion restreinte — mairie de Ratoma et prestataire

Sommaire

1. À qui s'adresse ce guide

2. Vue d'ensemble et choix du mode de déploiement

3. Prérequis

4. Gestion des secrets et des configurations

5. Variables d'environnement

6. Installation du backend (mode B)

- 6.1 Préparer le serveur et le code
- 6.2 Configurer l'environnement
- 6.3 Initialiser la base et lancer le service

7. Base de données PostgreSQL

8. Installation du frontend (mode B)

9. Serveur web, nom de domaine et HTTPS (mode B)

- 9.1 Nom de domaine
- 9.2 Configuration nginx
- 9.3 Certificat HTTPS

10. Migrations et premier administrateur

11. Déploiement sur Vercel et Neon (mode A, en service)

12. Sauvegarde

- 12.1 Commande fournie
- 12.2 Planification automatique
- 12.3 Quand et combien de temps conserver

13. Restauration après incident

- 13.1 Procédure
- 13.2 Situations courantes

14. Mises à jour

15. Journaux techniques et supervision

16. Dépannage

17. Liste de contrôle avant la mise en exploitation réelle

18. Registre d'exploitation

1. À qui s'adresse ce guide

Ce guide permet à un technicien de la Mairie de Ratoma, ou à tout prestataire qui reprendrait le système, d'installer, d'exploiter et de maintenir Wakara **sans dépendre du développeur initial**. Il suppose des connaissances générales en administration de serveurs et en ligne de commande.

> **Aucun mot de passe, clé ou jeton n'est écrit dans ce guide.** Les secrets se configurent par variables d'environnement ou dans le tableau de bord de l'hébergeur, jamais dans le code ni dans le dépôt Git (voir le chapitre 4).

2. Vue d'ensemble et choix du mode de déploiement

	A. Vercel + Neon (en service)	B. Serveur classique (Linux)
Principe	L'hébergeur construit et publie le site à chaque envoi de code ; la base est un service PostgreSQL géré	Vous installez et administrez un serveur : API (gunicorn), serveur web (nginx), PostgreSQL
Avantages	Aucun serveur à administrer ; HTTPS et déploiement automatiques ; mise en ligne immédiate	Maîtrise complète du lieu d'hébergement et des données ; pas de dépendance à un service étranger
Contraintes	Données hébergées chez des fournisseurs étrangers ; limites de l'offre gratuite ; service « Vercel Services » en bêta	Administration, sécurité et sauvegardes à votre charge
État	**Déployé et vérifié** (chapitre 10)	Procédure de référence ; **non éprouvée de bout en bout** dans l'environnement de développement (chapitres 5 à 8)

Les chapitres 4, 9, 11, 12, 13 et 14 s'appliquent aux deux modes.

3. Prérequis

Élément	Mode A (Vercel + Neon)	Mode B (serveur)
Poste d'administration	Navigateur, Git, Python 3.12+ (pour les commandes de maintenance)	Idem
Serveur	Aucun	Linux 64 bits, 2 Go de mémoire, 2 cœurs, 20 Go de disque (à dimensionner selon le volume de pièces)
Logiciels	Comptes Vercel, Neon et du dépôt de code	Python 3.12+, Node.js 18+ (construction de l'interface), PostgreSQL 15+, nginx (ou Caddy), certbot
Réseau	Accès Internet	Adresse IP publique, nom de domaine, ports 80 et 443 ouverts
Données	Aucune donnée réelle avant autorisation de la mairie	Idem

4. Gestion des secrets et des configurations

- Chaque environnement (développement, recette, production) a **sa propre configuration**, sa propre base et ses propres secrets.
- Les secrets se saisissent dans les variables d'environnement du service (onglet Environment Variables de Vercel, fichier `EnvironmentFile` de systemd, coffre de secrets). Le fichier `backend/.env` ne sert qu'au développement et **n'est jamais commité** (il est exclu de Git).
- Une clé exposée par erreur doit être **changée immédiatement** : générer une nouvelle valeur, la configurer, redéployer.
- Tout mot de passe initial est **à changer à la première connexion** ; l'application l'exige.
- Le mot de passe de l'administrateur initial (`WAKARA_ADMIN_PASSWORD`) n'est lu qu'à la création du compte : **supprimer cette variable** une fois l'administrateur connecté.
- Générer une clé secrète : `python -c "import secrets; print(secrets.token_urlsafe(64))"` .
- Conserver les secrets dans un gestionnaire de mots de passe de la mairie, jamais dans un courriel ni dans ce dossier.

5. Variables d'environnement

Variable	Valeur par défaut	Rôle
`SECRET_KEY`	— (obligatoire hors Vercel)	Clé de signature Django : longue, aléatoire, propre à chaque environnement
`DEBUG`	`False`	`True` uniquement en développement
`ALLOWED_HOSTS`	`localhost,127.0.0.1`	Noms d'hôte servis (domaine de production)
`CSRF_TRUSTED_ORIGINS`	vide	Origines HTTPS de confiance (ex. `https://wakara.exemple.gn`)
`CORS_ALLOWED_ORIGINS`	`http://localhost:4200`	Origines de l'interface (développement)
`DATABASE_URL`	vide	URL PostgreSQL complète avec `?sslmode=require` (remplace les `DB_*`)
`DB_ENGINE` , `DB_NAME` , `DB_USER` , `DB_PASSWORD` , `DB_HOST` , `DB_PORT`	SQLite	Connexion PostgreSQL détaillée (si `DATABASE_URL` n'est pas utilisée)
`JWT_ACCESS_TOKEN_LIFETIME_MINUTES`	`60`	Durée du jeton d'accès
`JWT_REFRESH_TOKEN_LIFETIME_DAYS`	`7`	Durée du jeton de rafraîchissement
`LOGIN_THROTTLE_RATE`	`10/min`	Limite des tentatives de connexion par adresse IP
`LOGIN_MAX_FAILED_ATTEMPTS` , `LOGIN_LOCKOUT_MINUTES`	`5` , `15`	Verrouillage temporaire d'un compte

Variable	Valeur par défaut	Rôle
`NUM_PROXIES`	`1` sur Vercel, `0` ailleurs	Nombre de relais de confiance devant Django (adresse IP du client)
`PRIVATE_MEDIA_ROOT`	`backend/private_media`	Dossier des pièces (mode fichiers), hors de tout dossier public
`DOCUMENTS_STORAGE`	`filesystem`, `database` sur Vercel	Où sont gardées les pièces : `filesystem`, `database` ou `s3`
`DOCUMENTS_S3_BUCKET`, `DOCUMENTS_S3_ACCESS_KEY`, `DOCUMENTS_S3_SECRET_KEY`, `DOCUMENTS_S3_ENDPOINT_URL`, `DOCUMENTS_S3_REGION`	vide	Stockage des pièces dans un bucket S3 **privé** (option)
`MAX_UPLOAD_SIZE_MB`	`10` (`4` sur Vercel)	Taille maximale d'une pièce
`WAKARA_ADMIN_USERNAME`, `WAKARA_ADMIN_PASSWORD`, `WAKARA_ADMIN_EMAIL`	—	Premier administrateur technique créé par `setup_initial_data`
`WAKARA_SECRETARY_*`, `WAKARA_AGENT_*`	—	Comptes de départ facultatifs (secrétaire, agent), même principe
`SECURE_SSL_REDIRECT`, `SECURE_HSTS_SECONDS`	`True`, `31536000`	Durcissement HTTPS (actif quand `DEBUG=False`)
`DECLARATION_REQUIRED_FOR_TRANSMISSION`, `ACTE_REQUIRED_FOR_SUBMISSION`, `ACTE_AUTO_NUMBERING`, `STATS_SHOW_GENDER`	voir le dossier technique	Hypothèses métier configurables

6. Installation du backend (mode B)

6.1 Préparer le serveur et le code

```
sudo apt update && sudo apt install -y python3 python3-venv python3-pip git nginx postgresql
sudo useradd --system --create-home --shell /usr/sbin/nologin wakara
sudo mkdir -p /opt/wakara && sudo chown wakara:wakara /opt/wakara
sudo -u wakara git clone https://github.com/alhassaneplea4/wakara-rtm1 /opt/wakara
cd /opt/wakara/backend
sudo -u wakara python3 -m venv venv
sudo -u wakara venv/bin/pip install -r requirements.txt gunicorn
```

6.2 Configurer l'environnement

Créer le fichier `/etc/wakara/wakara.env` lisible uniquement par le compte du service (`chmod 600`, propriétaire `wakara`), avec les variables du chapitre 4, par exemple :

```
SECRET_KEY=<valeur générée>
DEBUG=False
ALLOWED_HOSTS=wakara.exemple.gn
CSRF_TRUSTED_ORIGINS=https://wakara.exemple.gn
DATABASE_URL=<url postgresql>?sslmode=require
NUM_PROXIES=1
PRIVATE_MEDIA_ROOT=/var/lib/wakara/private_media
WAKARA_ADMIN_USERNAME=<identifiant>
WAKARA_ADMIN_PASSWORD=<mot de passe robuste>
```

Une seule valeur par ligne, **sans commentaire en fin de ligne**. Les variables `DB_NAME`, `DB_USER`, `DB_PASSWORD` et `DB_HOST` peuvent remplacer `DATABASE_URL`. `WAKARA_ADMIN_PASSWORD` sera supprimée après la première connexion de l'administrateur.

6.3 Initialiser la base et lancer le service

```
sudo mkdir -p /var/lib/wakara/private_media && sudo chown wakara:wakara /var/lib/wakara/private_media
&& sudo chmod 700 /var/lib/wakara/private_media
sudo -u wakara bash                       # ouvre un shell au nom du compte du service
set -a; . /etc/wakara/wakara.env; set +a  # charge les variables de la configuration
cd /opt/wakara/backend
venv/bin/python manage.py migrate
venv/bin/python manage.py collectstatic --noinput
venv/bin/python manage.py setup_initial_data
venv/bin/python manage.py check --deploy
exit
```

Service systemd `/etc/systemd/system/wakara.service` :

```
[Unit]
Description=Wakara API (gunicorn)
After=network.target postgresql.service

[Service]
User=wakara
WorkingDirectory=/opt/wakara/backend
EnvironmentFile=/etc/wakara/wakara.env
ExecStart=/opt/wakara/backend/venv/bin/gunicorn wakara.wsgi:application --bind 127.0.0.1:8000 --
workers 3
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload && sudo systemctl enable --now wakara
sudo systemctl status wakara
```

> Avec plusieurs processus gunicorn, remplacer le cache mémoire local par un cache partagé (Redis ou Memcached, réglage `CACHES`) pour que la limitation des tentatives de connexion soit commune à tous les processus.

7. Base de données PostgreSQL

Créer une base et un utilisateur dédiés à Wakara, avec un mot de passe robuste (remplacer les valeurs entre chevrons) :

```
sudo -u postgres psql
CREATE ROLE wakara_app LOGIN PASSWORD '<mot de passe robuste>';
CREATE DATABASE wakara OWNER wakara_app ENCODING 'UTF8';
\q
```

- Renseigner ensuite `DATABASE_URL` (`postgres://wakara_app:<mot de passe>@localhost:5432/wakara`) ou les variables `DB_*`.
- Avec un fournisseur géré (Neon par exemple), utiliser l'URL fournie, avec `?sslmode=require` .
- **Durcissement recommandé** : une fois les migrations appliquées, retirer à l'utilisateur applicatif le droit de modifier ou supprimer le journal : `REVOKE UPDATE, DELETE, TRUNCATE ON audit_auditlog, declarations_statushistory FROM wakara_app;` . Les migrations futures devront alors être exécutées par un compte distinct, propriétaire du schéma.
- N'ouvrir le port PostgreSQL (5432) à aucun réseau extérieur.

8. Installation du frontend (mode B)

```
cd /opt/wakara/frontend
npm ci
npm run build            # produit dist/wakara-frontend/browser
```

L'adresse de l'API se règle dans `frontend/src/environments/environment.prod.ts` : par défaut `apiUrl: '/api'`, c'est-à-dire l'API sur le même domaine. Ne la modifier que si l'API est servie ailleurs.

9. Serveur web, nom de domaine et HTTPS (mode B)

9.1 Nom de domaine

Créer chez le registraire un enregistrement DNS de type `A` (et `AAAA` si IPv6) qui pointe le nom choisi (par exemple `wakara.exemple.gn`) vers l'adresse IP publique du serveur. Attendre la propagation avant de demander le certificat.

9.2 Configuration nginx

Fichier `/etc/nginx/sites-available/wakara` (à adapter : nom de domaine et chemins) :

```
server {
    listen 80;
    server_name wakara.exemple.gn;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    http2 on;
    server_name wakara.exemple.gn;

    ssl_certificate     /etc/letsencrypt/live/wakara.exemple.gn/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/wakara.exemple.gn/privkey.pem;

    root /opt/wakara/frontend/dist/wakara-frontend/browser;
    client_max_body_size 12m;

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /static/ {
        alias /opt/wakara/backend/staticfiles/;
    }

    location / {
        try_files $uri /index.html;
    }
}
```

- **Ne jamais** servir le dossier `PRIVATE_MEDIA_ROOT` : les pièces ne sortent que par l'API, après authentification.
- Régler `NUM_PROXIES=1` : nginx est le relais de confiance et ajoute l'adresse du client à `X-Forwarded-For`.
- Si Django Admin n'est pas nécessaire, ne pas router `/admin/` ; il ne doit être ouvert qu'à un réseau de confiance.

9.3 Certificat HTTPS

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d wakara.exemple.gn
sudo nginx -t && sudo systemctl reload nginx
sudo certbot renew --dry-run     # vérifie le renouvellement automatique
```

10. Migrations et premier administrateur

1. Appliquer les migrations : `python manage.py migrate` (automatique à chaque déploiement sur Vercel).
2. Créer les données de référence et le premier administrateur : `python manage.py setup_initial_data`. La commande est **idempotente** : elle peut être relancée sans créer de doublon. Elle crée les 32 permissions, les

3 rôles, les paramètres, l'administrateur technique (rôle métier, **pas** superutilisateur Django) et, une seule fois, les établissements de Ratoma.

3. Se connecter avec l'identifiant `WAKARA_ADMIN_USERNAME` et le mot de passe fourni : l'application exige immédiatement d'en choisir un nouveau.
4. Supprimer `WAKARA_ADMIN_PASSWORD` de la configuration.
5. Créer ensuite, depuis l'écran « Utilisateurs », les comptes des secrétaires et des agents de collecte.
6. Pour un accès à l'administration Django (maintenance uniquement) : `python manage.py createsuperuser` ; ce compte n'a aucun droit métier.

> Ne jamais lancer `seed_demo` en production : la commande le refuse, c'est voulu. Les établissements préremplis sont une proposition à valider par la mairie.

11. Déploiement sur Vercel et Neon (mode A, en service)

1. **Dépôt** : le code est sur GitHub (`https://github.com/alhassaneplea4/wakara-rtm1`). Chaque envoi sur la branche `main` déclenche un déploiement.
2. **Projet Vercel** : créer ou ouvrir le projet, relier le dépôt, choisir le préréglage de framework « Services ». Le fichier `vercel.json` déclare le service `backend` (dossier `backend`, Django, commande de build `python build.py`) et le service `frontend` (dossier `frontend`, Angular), avec les redirections `/api/*` vers le backend et le reste vers l'interface.
3. **Base** : dans l'onglet Storage (ou Marketplace) du projet, créer une base **Neon** et la connecter ; Vercel ajoute la variable `DATABASE_URL` .
4. **Variables** (Settings → Environment Variables, environnement Production) : `WAKARA_ADMIN_USERNAME` et `WAKARA_ADMIN_PASSWORD` (type secret) ; facultatives : `SECRET_KEY`, comptes de départ, `ALLOWED_HOSTS` et `CSRF_TRUSTED_ORIGINS` pour un domaine personnalisé.
5. **Premier déploiement** : l'étape de build exécute `migrate` puis `setup_initial_data` (production uniquement). Elle échoue avec un message explicite si `DATABASE_URL` est absente.
6. **Vérification** : ouvrir `https://wakara-rtm.vercel.app/api/public/settings/` (doit répondre en JSON), puis la page de connexion, puis se connecter ; créer une déclaration avec une photo et la rouvrir.
7. **Domaine personnalisé** : ajouter le domaine dans Vercel (Settings → Domains), suivre les instructions DNS, puis renseigner `ALLOWED_HOSTS` et `CSRF_TRUSTED_ORIGINS` .
8. **Après la mise en service** : supprimer `WAKARA_ADMIN_PASSWORD` de Vercel.

Les déploiements de prévisualisation (branches autres que `main`) n'appliquent jamais de migration et ne touchent pas à la base de production.

12. Sauvegarde

Deux éléments se sauvegardent **ensemble** : la base de données et les pièces justificatives. Sur Vercel avec le stockage en base, les pièces sont dans la base : une seule sauvegarde suffit.

12.1 Commande fournie

```
sudo -u wakara bash
set -a; . /etc/wakara/wakara.env; set +a
cd /opt/wakara/backend
venv/bin/python manage.py backup_wakara --output-dir /var/backups/wakara --keep 30
exit
```

La commande produit `wakara_AAAAMMJJ_HHMMSS.zip` : la base (SQLite : copie cohérente ; PostgreSQL : `pg_dump` au format personnalisé, mot de passe fourni par `PGPASSWORD`) et le dossier des pièces. Les archives au-delà de `--keep` sont supprimées, les plus anciennes d'abord. Avec un stockage S3, le bucket n'est pas inclus : activer le versionnement du bucket.

12.2 Planification automatique

Linux, chaque nuit à 2 h : fichier `/etc/cron.d/wakara` (les dossiers `/var/backups/wakara` et `/var/log/wakara` doivent exister et appartenir au compte `wakara`).

```
0 2 * * * wakara bash -c 'set -a; . /etc/wakara/wakara.env; set +a; cd /opt/wakara/backend &&
venv/bin/python manage.py backup_wakara --output-dir /var/backups/wakara --keep 30 >>
/var/log/wakara/backup.log 2>&1'
```

Copier les archives **hors du serveur** (autre site, stockage chiffré). Pour Neon, activer et vérifier la fenêtre de restauration à un instant donné et les sauvegardes du fournisseur ; elles ne remplacent pas une copie conservée par la mairie.

12.3 Quand et combien de temps conserver

- Sauvegarde quotidienne, conservation de 30 jours au minimum ; une archive mensuelle conservée plus longtemps.
- Durées légales de conservation : à confirmer par la mairie (décision n° 7 du cahier des charges).

13. Restauration après incident

13.1 Procédure

1. **Arrêter** l'application (ou suspendre l'accès) pour éviter de nouvelles écritures.
2. Choisir l'archive à restaurer (la plus récente saine).
3. Lancer `python manage.py restore_wakara <archive.zip> --yes`. SQLite : le fichier courant est déplacé en `*.before-restore-<date>`, jamais supprimé. PostgreSQL : le dump est extrait dans `backend/restore_<date>.dump` et la commande `pg_restore` à exécuter est affichée (elle exige les droits d'administration de la base). Les pièces sont restaurées et l'ancien dossier mis de côté.
4. Lancer `python manage.py check`, puis redémarrer le service.
5. **Vérifier** : connexion, ouverture d'un dossier récent, téléchargement d'une pièce, présence du dernier acte validé.
6. Consigner l'incident et la restauration dans le registre d'exploitation.

13.2 Situations courantes

Situation	Conduite à tenir
Le site ne répond plus après un déploiement	Sur Vercel : ouvrir le déploiement en erreur, lire le journal de build, puis rétablir le déploiement précédent depuis la liste des déploiements. Mode B : `journalctl -u wakara -n 100`
Une migration a échoué	Ne pas modifier la base à la main. Lire l'erreur, corriger le code ou la donnée en cause, relancer. En cas de doute, restaurer la dernière sauvegarde et rétablir la version précédente du code
La base est corrompue ou des données ont été perdues	Restaurer la dernière archive saine (procédure ci-dessus) ; Neon permet aussi une restauration à un instant donné
Le mot de passe administrateur est perdu	Depuis le poste d'administration, avec les variables de production : créer un nouvel administrateur par `setup_initial_data` avec un nouvel identifiant, ou réinitialiser le compte par un autre administrateur
Un secret a été exposé	Changer le secret (clé Django, mot de passe de base, jeton), redéployer, puis vérifier le journal d'activité
Compte compromis	Désactiver le compte depuis « Utilisateurs » (ses jetons sont révoqués immédiatement), réinitialiser son mot de passe, consulter son activité dans le journal

> **Une sauvegarde non testée n'est pas une sauvegarde.** Le test automatique vérifie l'aller-retour sur SQLite. Il faut **restaurer au moins une fois une archive réelle PostgreSQL sur un serveur de test** et consigner la date et le résultat (scénario E5 du procès-verbal de recette).

14. Mises à jour

1. Lire les notes de version et sauvegarder la base avant toute mise à jour.
2. Récupérer le nouveau code : `git pull` (mode B) ou envoi sur `main` (mode A).
3. Mode B : `venv/bin/pip install -r requirements.txt`, `manage.py migrate`, `manage.py collectstatic --noinput`, reconstruire l'interface (`npm ci && npm run build`), puis `sudo systemctl restart wakara`.
4. Contrôler : connexion, un dossier, une impression d'acte, le tableau de bord.
5. En cas de problème : revenir au commit précédent et, si une migration a modifié les données, restaurer la sauvegarde.

Avant d'envoyer du code sur `main`, lancer l'ensemble des contrôles : `python manage.py test tests`, `python manage.py makemigrations --check`, `npm run build`, `npx tsc -p tsconfig.spec.json --noEmit`.

15. Journaux techniques et supervision

Besoin	Où regarder
Erreurs du serveur d'API (mode A)	Vercel → projet → Logs (filtrer sur le service `backend`)
Erreurs du serveur d'API (mode B)	`journalctl -u wakara` ; journaux nginx dans `/var/log/nginx/`
Échec d'un déploiement	Vercel → Deployments → journal de build

Besoin	Où regarder
Qui a fait quoi dans l'application	Écrans « Journaux d'activité » et « Historiques des actes » (administrateur)
Santé de la base	Tableau de bord du fournisseur (Neon) : stockage utilisé, connexions
Disponibilité du site	Vérifier `/api/public/settings/` (réponse JSON) ; mettre en place une sonde externe si possible

16. Dépannage

Symptôme	Cause probable	Remède
« No module named django »	Le Python global est utilisé au lieu de l'environnement virtuel	Lancer avec `venv/bin/python` (ou `backend\venv\Scripts\python.exe` sous Windows)
Presque tous les tests échouent en local	Redirection HTTPS active	Lancer avec `DEBUG=True` ou `SECURE_SSL_REDIRECT=False`
Erreur 400 « Invalid host header »	`ALLOWED_HOSTS` ne contient pas le domaine	Ajouter le domaine à `ALLOWED_HOSTS`
Connexion impossible depuis le domaine personnalisé (403 CSRF)	`CSRF_TRUSTED_ORIGINS` incomplet	Ajouter `https://<domaine>`
Menu inactif, déconnexion sans effet	Onglet resté ouvert pendant une mise à jour du site	Recharger la page (Ctrl+F5)
Envoi d'une photo refusé	Fichier trop lourd (4 Mo sur Vercel) ou format non accepté	Réduire la photo ; formats acceptés : JPG, PNG, PDF
Le build de production échoue : « DATABASE_URL est absente »	Base non connectée au projet	Connecter la base Neon ou renseigner `DATABASE_URL`
Adresse IP du journal d'audit incorrecte	`NUM_PROXIES` mal réglé	`1` derrière un seul relais, `0` sans relais

17. Liste de contrôle avant la mise en exploitation réelle

	Contrôle	Date / visa
☐	`python manage.py check --deploy` sans alerte bloquante	
☐	Mot de passe de l'administrateur initial changé ; variable `WAKARA_ADMIN_PASSWORD` supprimée ; aucun compte de démonstration	
☐	HTTPS valide et renouvellement automatique du certificat vérifié	
☐	Sauvegarde automatique planifiée, copie hors site, **restauration testée sur PostgreSQL**	
☐	Droits de modification du journal retirés à l'utilisateur applicatif (`REVOKE`)	
☐	Aucun secret dans le dépôt ni dans les documents remis	

	Contrôle	Date / visa
☐	Décisions métier validées par la mairie : numérotation, modèle d'acte, hébergement, conservation	
☐	Liste des établissements préremplis relue et corrigée	
☐	Pièces d'état civil inaccessibles sans connexion (test manuel d'une adresse de pièce sans jeton)	
☐	Utilisateurs formés ; comptes réels créés avec mot de passe temporaire	

18. Registre d'exploitation

À tenir par l'administrateur technique : chaque intervention significative (mise à jour, incident, restauration, changement de secret, changement de droits) y est consignée.

Date	Intervention	Auteur	Résultat	Remarques